

Flyweight Design Pattern

Intent

Use sharing to support large numbers of fine-grained objects efficiently. Instead of creating one object per logical use, split each object's state in two:

- **Intrinsic state** — immutable, shareable, independent of context (e.g. a seat class's cabin tier). This lives *inside* a small pool of flyweight objects.
- **Extrinsic state** — context-specific, varies per use (e.g. *which* seat number and *which* passenger). This is never stored on the flyweight; it is supplied by the caller at the point of use.

A factory hands out flyweights keyed by identity: the first request for a key builds the object and caches it, every later request for the same key returns the **same** instance. Any number of logical reservations then costs only one object per distinct key.

UML class diagram



The players

GoF role	Class in this module
Flyweight (interface)	contract/ISeatClass — declares <code>reserve(String seatNumber, String passengerName)</code>
ConcreteFlyweight (shared instances)	contract/concrete/EconomySeatClass (cabin tier ECONOMY), contract/concrete/BusinessSeatClass (cabin tier BUSINESS)
Flyweight key	contract/enums/SeatClassType — ECONOMY, BUSINESS
FlyweightFactory (creates + pools)	factory/SeatClassFactory — lazily fills an <code>EnumMap<SeatClassType, ISeatClass></code> via <code>computeIfAbsent</code>
Client	SeatBookingDesignPattern.main()

Code walkthrough

contract/ISeatClass.java — the Flyweight interface

```
package com.design.patterns.flyweight.contract;

public interface ISeatClass {

    void reserve(String seatNumber, String passengerName);

}
```

- `package com.design.patterns.flyweight.contract;` — the interface lives in a `contract` package, separate from its implementations (`contract.concrete`) and from the factory that assembles them; this keeps the abstraction importable without pulling in any concrete class.
- `public interface ISeatClass` — this is the **Flyweight** role: the one type every client and the factory depend on. Nothing outside this module ever needs to know `EconomySeatClass` or `BusinessSeatClass` exist.
- `void reserve(String seatNumber, String passengerName);` — the single operation a flyweight supports, and it takes the **extrinsic state** (`seatNumber`, `passengerName` — who is sitting where) as method parameters. This is the mechanical proof that a reservation's specifics are not part of the object: they cannot be, because the interface never exposes a way to set them — they only ever arrive as arguments, fresh on every call.

contract/concrete/EconomySeatClass.java and contract/concrete/BusinessSeatClass.java — ConcreteFlyweights

```
package com.design.patterns.flyweight.contract.concrete;

import com.design.patterns.flyweight.contract.ISeatClass;

public class EconomySeatClass implements ISeatClass {

    private final String cabinTier = "ECONOMY";

    @Override
    public void reserve(String seatNumber, String passengerName) {
        System.out.println("---Reserving " + cabinTier + " seat " + seatNumber + " for " + passengerName);
    }

}
```

- `package com.design.patterns.flyweight.contract.concrete;` — sits under `contract`, alongside the interface it implements, and is named `concrete` (not abbreviated) to say plainly what it holds: the concrete implementations of `ISeatClass`.
- `private final String cabinTier = "ECONOMY";` — this is the **intrinsic state**. It is `private` (no setter, no way for a caller to reach in), `final` (cannot change after construction), and initialized once from a literal. Because it can never change and is never told apart per reservation, it is exactly the kind of state that is safe to share across every reservation made against a `EconomySeatClass`.
- `public void reserve(String seatNumber, String passengerName)` — implements the Flyweight contract. Note `cabinTier` (intrinsic, from the field) and `seatNumber` / `passengerName` (extrinsic, from the parameters) are combined only inside this method body, at the moment of use — they are never merged into shared mutable state.
- `System.out.println(...)` — the "confirmation" stand-in for this module; a real booking engine would persist the reservation and emit an itinerary using the shared tier rules (baggage allowance, boarding priority, etc.).

`BusinessSeatClass` is structurally identical, with `cabinTier = "BUSINESS"` — a second ConcreteFlyweight sharing the exact same contract, proving the factory/pool mechanism generalizes to more than one flyweight type.

contract/enums/SeatClassType.java — the flyweight key

```
package com.design.patterns.flyweight.contract.enums;

public enum SeatClassType {

    ECONOMY,

    BUSINESS;

}
```

- A closed, type-safe key set. Using an `enum` (rather than a raw `String`) means the factory's pool can be an `EnumMap` — the cheapest, fastest possible flyweight pool — and means every valid key is known and exhaustively `switch`-able at compile time; there is no way to mistype a key and silently miss the pool.

factory/SeatClassFactory.java — the FlyweightFactory

```
package com.design.patterns.flyweight.factory;

import java.util.EnumMap;
import java.util.Map;
```

```

import com.design.patterns.flyweight.contract.ISeatClass;
import com.design.patterns.flyweight.contract.concrete.BusinessSeatClass;
import com.design.patterns.flyweight.contract.concrete.EconomySeatClass;
import com.design.patterns.flyweight.contract.enums.SeatClassType;

public class SeatClassFactory {

    private final Map<SeatClassType, ISeatClass> pool = new EnumMap<>(SeatClassType.class);

    public ISeatClass getSeatClass(SeatClassType type) {
        return pool.computeIfAbsent(type, SeatClassFactory::create);
    }

    private static ISeatClass create(SeatClassType type) {
        System.out.println("(pool miss) creating flyweight for " + type);
        switch (type) {
            case ECONOMY:
                return new EconomySeatClass();
            case BUSINESS:
                return new BusinessSeatClass();
            default:
                throw new IllegalArgumentException("No flyweight registered for " + type);
        }
    }

    public int poolSize() {
        return pool.size();
    }
}

```

- `private final Map<SeatClassType, ISeatClass> pool = new EnumMap<>(SeatClassType.class);` — this is **the pool**: the one piece of mutable state the whole pattern hinges on. It is a field on the factory (not local to a method), so it survives across calls and accumulates shared instances over the factory's lifetime. `EnumMap` is used instead of `HashMap` because the key domain (`SeatClassType`) is a closed enum — it is both faster and self-documenting about the key space.
- `public ISeatClass getSeatClass(SeatClassType type)` — the **one and only way** a client obtains an `ISeatClass`. There is no public constructor exposed on `EconomySeatClass` / `BusinessSeatClass` usage path that a client is meant to call directly; going through the factory is the contract.
- `pool.computeIfAbsent(type, SeatClassFactory::create)` — this line **is** the sharing mechanism. `computeIfAbsent` checks the map first: if `type` is already a key, it returns the existing value immediately and `create` is never invoked; only on a genuine miss does it call `create(type)`, store the result, and return it. This is a real cache, not `new X()` on every call — the same `ISeatClass` reference is handed out to every subsequent caller asking for the same `type`.
- `private static ISeatClass create(SeatClassType type)` — factored out from `getSeatClass` so the "pool miss" log line and the `switch` that actually builds a flyweight are visible and testable in isolation. It is `static` because building a flyweight needs no factory instance state — only the requested `type`.
- `System.out.println("(pool miss) creating flyweight for " + type);` — deliberately logs every real construction, so the demo (and this README's captured output) can prove sharing by counting how many times this line fires versus how many times `getSeatClass` is called.
- `switch (type) { case ECONOMY: return new EconomySeatClass(); case BUSINESS: return new BusinessSeatClass(); default: throw ... }` — the only two places `new` is called on a concrete flyweight in the entire module. The `default` throws rather than returning `null`, so a future `SeatClassType` constant added without a matching `case` fails loudly at first use instead of producing a silent `NullPointerException` deeper in client code.
- `public int poolSize()` — exposes the pool's current size so a caller (here, the demo) can assert how many real objects exist versus how many logical reservations were requested — the whole point of the pattern made observable.

SeatBookingDesignPattern.java — the client

```

package com.design.patterns.flyweight;

import com.design.patterns.flyweight.contract.ISeatClass;
import com.design.patterns.flyweight.contract.enums.SeatClassType;
import com.design.patterns.flyweight.factory.SeatClassFactory;

public class SeatBookingDesignPattern {

    public static void main(String[] args) {
        System.out.println("Flyweight Design Pattern");

        SeatClassFactory seatClassFactory = new SeatClassFactory();

        ISeatClass booking1 = seatClassFactory.getSeatClass(SeatClassType.ECONOMY);
        booking1.reserve("14A", "A. Menon");
        ISeatClass booking2 = seatClassFactory.getSeatClass(SeatClassType.ECONOMY);
        booking2.reserve("22F", "R. Iyer");
        seatClassFactory.getSeatClass(SeatClassType.BUSINESS).reserve("2C", "S. Kapoor");
        seatClassFactory.getSeatClass(SeatClassType.BUSINESS).reserve("3A", "N. Verma");

        System.out.println("ECONOMY flyweight reused (booking1 == booking2): " + (booking1 == booking2));
        System.out.println("Objects in pool for 4 reservations: " + seatClassFactory.poolSize());
    }
}

```

```
}
}
```

- `SeatClassFactory seatClassFactory = new SeatClassFactory();` — the client owns exactly one factory instance; every seat class it needs comes from this one pool.
- `ISeatClass booking1 = seatClassFactory.getSeatClass(SeatClassType.ECONOMY);` followed later by `ISeatClass booking2 = seatClassFactory.getSeatClass(SeatClassType.ECONOMY);` — two separate logical "reservations" against an economy seat class, requested independently.
- `booking1.reserve("14A", "A. Menon");` / `booking2.reserve("22F", "R. Iyer");` — the extrinsic reservation details are supplied here, at the call site, once per reservation — never at construction time and never stored anywhere between calls.
- `seatClassFactory.getSeatClass(SeatClassType.BUSINESS).reserve("2C", "S. Kapoor");` and the line after it — two more reservations, this time against the other flyweight type, each with its own extrinsic details, chained without holding a local variable at all (nothing about the pattern requires keeping a reference).
- `System.out.println("ECONOMY flyweight reused (booking1 == booking2): " + (booking1 == booking2));` — the load-bearing assertion of the whole module: `==` compares **object identity**, not equality of contents. If the factory were not sharing, `booking1` and `booking2` would be two distinct `EconomySeatClass` instances and this would print `false`. Printing `true` is direct proof the same object was handed back on the second request.
- `System.out.println("Objects in pool for 4 reservations: " + seatClassFactory.poolSize());` — the second proof: four `getSeatClass` / `reserve` calls happened, but the pool holds only 2 objects (one `EconomySeatClass`, one `BusinessSeatClass`) — the pattern's efficiency claim made concrete and countable.

Why these design decisions

- **Why `computeIfAbsent` over a manual `if (!pool.containsKey(...))` check?** It is atomic and idiomatic: one call expresses "get or create and store," with no window where a caller could observe a half-populated pool. A hand-rolled `get-then-put` is exactly the kind of thing that quietly rots into a double-construction bug under any future refactor (e.g. if the factory ever became concurrent).
- **Why is `cabinTier` a field on the flyweight, but `(seatNumber, passengerName)` a method parameter — why not both fields, or both parameters?** Because they differ in shareability. `cabinTier` is the same for every `EconomySeatClass` that will ever exist, so baking it into the shared instance costs nothing and lets every caller benefit from the one cached object. `(seatNumber, passengerName)` is different on every reservation; if it were a field, every reservation would need its own object, destroying the sharing the pattern exists to provide. The two are separated exactly along the intrinsic/extrinsic line the GoF pattern defines.
- **Why route every construction through `SeatClassFactory.create`, including the `switch`, instead of letting client code call `new EconomySeatClass()` directly?** A flyweight pattern is only as strong as its enforcement. If clients can bypass the factory, nothing stops the pool from becoming inconsistent (multiple non-shared instances for the same key). Keeping `new` confined to one private static method inside the factory makes the factory the sole authority over identity.
- **Why `EnumMap<SeatClassType, ISeatClass>` instead of `HashMap<String, ISeatClass>`?** The key space is closed and known at compile time, so an enum key gives compiler-checked exhaustiveness (via the `switch` in `create`) and `EnumMap`'s array-backed storage is faster and more memory-compact than hashing a `String` on every lookup — the fastest reasonable implementation of "a small pool keyed by identity."
- **Why does `create` throw on default instead of returning `null`?** A flyweight factory silently returning `null` for an unrecognized key turns a programming error into a `NullPointerException` far away from its cause, likely inside `reserve()`. Throwing `IllegalArgumentException` at the factory boundary fails fast, at the point where the real mistake (an unhandled `SeatClassType`) was made.

Execution flow trace

SeatBookingDesignPattern.main

```

├─ new SeatClassFactory()                                pool = {} (empty EnumMap)
├─ seatClassFactory.getSeatClass(ECONOMY) → pool miss → create(ECONOMY) → new EconomySeatClass() →
pool = {ECONOMY: economyA}
├─ booking1.reserve("14A", "A. Menon") → "----Reserving ECONOMY seat 14A for A. Menon"
├─ seatClassFactory.getSeatClass(ECONOMY) → pool HIT → returns economyA (no create() call, no log line)
├─ booking2.reserve("22F", "R. Iyer") → "----Reserving ECONOMY seat 22F for R. Iyer"
├─ seatClassFactory.getSeatClass(BUSINESS) → pool miss → create(BUSINESS) → new BusinessSeatClass() → pool
= {ECONOMY: economyA, BUSINESS: businessA}
├─ .reserve("2C", "S. Kapoor") → "----Reserving BUSINESS seat 2C for S. Kapoor"
├─ seatClassFactory.getSeatClass(BUSINESS) → pool HIT → returns businessA
├─ .reserve("3A", "N. Verma") → "----Reserving BUSINESS seat 3A for N. Verma"
├─ booking1 == booking2 → true (both reference economyA)
└─ seatClassFactory.poolSize() → 2 (economyA, businessA – for 4 reservations)

```

Expected output

Captured from a real run — `java -cp target/classes com.design.patterns.flyweight.SeatBookingDesignPattern`:

```
Flyweight Design Pattern
(pool miss) creating flyweight for ECONOMY
----Reserving ECONOMY seat 14A for A. Menon
----Reserving ECONOMY seat 22F for R. Iyer
(pool miss) creating flyweight for BUSINESS
----Reserving BUSINESS seat 2C for S. Kapoor
----Reserving BUSINESS seat 3A for N. Verma
ECONOMY flyweight reused (booking1 == booking2): true
Objects in pool for 4 reservations: 2
```

Only two `(pool miss)` lines appear despite four `getSeatClass` calls, and `booking1 == booking2` is `true` — the same object, not an equal copy. Four logical reservations, two real objects.

How to run

From inside this module directory:

```
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean compile
java -cp target/classes com.design.patterns.flyweight.SeatBookingDesignPattern
```